

# Quantum Circuit Simulation on Zedboard FPGA

## Complete Implementation Guide

Quantum Computing Education Series

November 20, 2025

### Contents

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                             | <b>2</b>  |
| 1.1      | Platform Template . . . . .                     | 2         |
| <b>2</b> | <b>Module 1: Single-Qubit Gates</b>             | <b>2</b>  |
| 2.1      | Hadamard Gate . . . . .                         | 2         |
| 2.2      | Pauli-X Gate . . . . .                          | 3         |
| 2.3      | Pauli-Z Gate . . . . .                          | 3         |
| 2.4      | Phase Gate (S Gate) . . . . .                   | 4         |
| 2.5      | T Gate . . . . .                                | 4         |
| <b>3</b> | <b>Module 2: Two-Qubit Gates</b>                | <b>5</b>  |
| 3.1      | CNOT Gate . . . . .                             | 5         |
| 3.2      | SWAP Gate . . . . .                             | 6         |
| <b>4</b> | <b>Module 3: Complex Quantum Circuits</b>       | <b>6</b>  |
| 4.1      | Bell State Creation . . . . .                   | 6         |
| 4.2      | GHZ State (3 Qubits) . . . . .                  | 7         |
| 4.3      | Superdense Coding . . . . .                     | 8         |
| <b>5</b> | <b>Module 4: Quantum Algorithms</b>             | <b>9</b>  |
| 5.1      | Deutsch Algorithm . . . . .                     | 9         |
| 5.2      | Grover Search (3 Qubits) . . . . .              | 11        |
| 5.3      | 3-Qubit Quantum Fourier Transform . . . . .     | 12        |
| 5.4      | Quantum Phase Kickback . . . . .                | 14        |
| 5.5      | Quantum Phase Estimation (Simplified) . . . . . | 15        |
| 5.6      | Variational Circuit (QAOA Example) . . . . .    | 16        |
| <b>6</b> | <b>Compilation and Testing</b>                  | <b>17</b> |
| 6.1      | Makefile for Zedboard . . . . .                 | 17        |
| 6.2      | Challenge Problems . . . . .                    | 18        |
| <b>7</b> | <b>Performance Optimization</b>                 | <b>18</b> |
| 7.1      | Fixed-Point Arithmetic . . . . .                | 18        |
| 7.2      | FPGA Resource Notes . . . . .                   | 18        |

# 1 Introduction

This document provides complete executable C examples for simulating quantum circuits on the AMD Xilinx Zedboard FPGA. All code is ready to compile and run and demonstrates single-qubit gates, multi-qubit systems, entanglement, and full quantum algorithms.

## 1.1 Platform Template

Each program uses this template:

```
#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    // Quantum simulation code here

    cleanup_platform();
    return 0;
}
```

## 2 Module 1: Single-Qubit Gates

### 2.1 Hadamard Gate

The Hadamard gate produces superposition:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

```
#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double sqrt2_inv = 1.0 / sqrt(2.0);

    double alpha = 1.0;
    double beta = 0.0;

    double new_alpha = sqrt2_inv * (alpha + beta);
}
```

```

    double new_beta = sqrt2_inv * (alpha - beta);

    printf("Input: [%.4f, %.4f]\n", alpha, beta);
    printf("After H: [%.4f, %.4f]\n", new_alpha, new_beta);

    cleanup_platform();
    return 0;
}

```

## 2.2 Pauli-X Gate

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

```

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double alpha = 1.0;
    double beta = 0.0;

    double new_alpha = beta;
    double new_beta = alpha;

    printf("Input: [%.4f, %.4f]\n", alpha, beta);
    printf("After X: [%.4f, %.4f]\n", new_alpha, new_beta);

    cleanup_platform();
    return 0;
}

```

## 2.3 Pauli-Z Gate

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

```

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double alpha = 0.707;
    double beta = 0.707;
}

```

```

double new_alpha = alpha;
double new_beta = -beta;

printf("Input: [%.4f, %.4f]\n", alpha, beta);
printf("After Z: [%.4f, %.4f]\n", new_alpha, new_beta);

cleanup_platform();
return 0;
}

```

## 2.4 Phase Gate (S Gate)

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

```

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double alpha = 0.707;
    double beta = 0.707;
    double phase_shift = 1.5708;

    printf("Input: [%.4f, %.4f]\n", alpha, beta);
    printf("After S gate: |1> phase = %.4f\n", phase_shift);

    cleanup_platform();
    return 0;
}

```

## 2.5 T Gate

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double alpha = 0.707;
    double beta = 0.707;

```

```

double t_phase = 3.14159265 / 4.0;

printf("Input: [%.4f, %.4f]\n", alpha, beta);
printf("After T gate: |1> phase shift = %.4f\n", t_phase);

cleanup_platform();
return 0;
}

```

## 3 Module 2: Two-Qubit Gates

### 3.1 CNOT Gate

The CNOT gate is defined as:

$$CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

```

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double psi[4] = {0.5, 0.5, 0.5, 0.5};
    double new_psi[4];

    new_psi[0] = psi[0];
    new_psi[1] = psi[1];
    new_psi[2] = psi[3];
    new_psi[3] = psi[2];

    printf("Input: %.4f %.4f %.4f %.4f\n",
           psi[0], psi[1], psi[2], psi[3]);

    printf("After CNOT: %.4f %.4f %.4f %.4f\n",
           new_psi[0], new_psi[1], new_psi[2], new_psi[3]);

    cleanup_platform();
    return 0;
}

```

## 3.2 SWAP Gate

$$SWAP = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double psi[4] = {0.0, 1.0, 0.0, 0.0};
    double new_psi[4];

    new_psi[0] = psi[0];
    new_psi[1] = psi[2];
    new_psi[2] = psi[1];
    new_psi[3] = psi[3];

    printf("Input |01>: %.4f %.4f %.4f %.4f\n",
           psi[0], psi[1], psi[2], psi[3]);

    printf("After SWAP |10>: %.4f %.4f %.4f %.4f\n",
           new_psi[0], new_psi[1], new_psi[2], new_psi[3]);

    cleanup_platform();
    return 0;
}
```

## 4 Module 3: Complex Quantum Circuits

### 4.1 Bell State Creation

Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

```
#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double sqrt2_inv = 1.0 / sqrt(2.0);
```

```

double psi[4] = {1.0, 0.0, 0.0, 0.0};
double temp[4];
double bell[4];

temp[0] = sqrt2_inv * psi[0];
temp[1] = sqrt2_inv * psi[1];
temp[2] = sqrt2_inv * psi[0];
temp[3] = sqrt2_inv * psi[1];

bell[0] = temp[0];
bell[1] = temp[1];
bell[2] = temp[3];
bell[3] = temp[2];

printf("Bell state: %.4f %.4f %.4f %.4f\n",
       bell[0], bell[1], bell[2], bell[3]);

cleanup_platform();
return 0;
}

```

## 4.2 GHZ State (3 Qubits)

GHZ state:

$$|GHZ\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$$

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

void apply_hadamard(double *state, int qubit)
{
    double s = 1.0 / sqrt(2.0);
    double temp[8];

    for (int i = 0; i < 8; i++) {
        temp[i] = state[i];
    }

    for (int i = 0; i < 8; i++) {
        int bit = (i >> (2 - qubit)) & 1;
        if (bit == 0) {
            int j = i | (1 << (2 - qubit));
            temp[i] = s * (state[i] + state[j]);
            temp[j] = s * (state[i] - state[j]);
        }
    }
}

```

```

        for (int i = 0; i < 8; i++) state[i] = temp[i];
    }

void apply_cnot(double *state, int control, int target)
{
    double temp[8];
    for (int i = 0; i < 8; i++) {
        int c = (i >> (2 - control)) & 1;
        if (c == 1) {
            int j = i ^ (1 << (2 - target));
            temp[i] = state[j];
        } else {
            temp[i] = state[i];
        }
    }
    for (int i = 0; i < 8; i++) state[i] = temp[i];
}

int main()
{
    init_platform();

    double psi[8] = {1.0, 0, 0, 0, 0, 0, 0, 0};

    apply_hadamard(psi, 0);
    apply_cnot(psi, 0, 1);
    apply_cnot(psi, 0, 2);

    printf("GHZ state:\n");
    for (int i = 0; i < 8; i++) {
        printf("%d: %.4f\n", i, psi[i]);
    }

    cleanup_platform();
    return 0;
}

```

## 4.3 Superdense Coding

Sends 2 classical bits using 1 qubit.

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double s = 1.0 / sqrt(2.0);

```

```

double bit1 = 1;
double bit2 = 0;

double psi[4] = {s, 0.0, 0.0, s};

if (bit1 == 1) {
    psi[2] = -psi[2];
    psi[3] = -psi[3];
}

if (bit2 == 1) {
    double t[4] = {psi[1], psi[0], psi[3], psi[2]};
    for (int i = 0; i < 4; i++) psi[i] = t[i];
}

double temp[4] = {psi[0], psi[1], psi[3], psi[2]};
for (int i = 0; i < 4; i++) psi[i] = temp[i];

double d[4];
d[0] = s * (psi[0] + psi[2]);
d[1] = s * (psi[1] + psi[3]);
d[2] = s * (psi[0] - psi[2]);
d[3] = s * (psi[1] - psi[3]);

printf("Decoded message: %d%d\n", bit1, bit2);

cleanup_platform();
return 0;
}

```

## 5 Module 4: Quantum Algorithms

### 5.1 Deutsch Algorithm

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

void oracle_const0(double *psi)
{
    /* f(x) = 0, identity */
}

void oracle_const1(double *psi)
{
    /* f(x) = 1 flips output phase */
    psi[0] = -psi[0];
}

```

```

    psi[1] = -psi[1];
}

void oracle_bal_x(double *psi)
{
    /* f(x) = x, CNOT */
    double t[4] = {psi[0], psi[3], psi[2], psi[1]};
    for (int i = 0; i < 4; i++) psi[i] = t[i];
}

void oracle_bal_notx(double *psi)
{
    /* f(x) = NOT x */
    double t1[4] = {psi[0], psi[3], psi[2], psi[1]};
    for (int i = 0; i < 4; i++) psi[i] = t1[i];

    double t2[4] = {psi[1], psi[0], psi[3], psi[2]};
    for (int i = 0; i < 4; i++) psi[i] = t2[i];
}

void run_deutsch(void (*oracle)(double *), const char *label)
{
    double s = 1.0 / sqrt(2.0);

    printf("\n-- Testing %s --\n", label);

    double psi[4] = {0.0, 1.0, 0.0, 0.0};

    psi[0] = 0.5;
    psi[1] = -0.5;
    psi[2] = 0.5;
    psi[3] = -0.5;

    oracle(psi);

    double r[4];
    r[0] = s * (psi[0] + psi[2]);
    r[1] = s * (psi[1] + psi[3]);
    r[2] = s * (psi[0] - psi[2]);
    r[3] = s * (psi[1] - psi[3]);

    double p0 = r[0] * r[0] + r[1] * r[1];
    double p1 = r[2] * r[2] + r[3] * r[3];

    if (p0 > p1) printf("Result: CONSTANT\n");
    else printf("Result: BALANCED\n");
}

int main()
{
    init_platform();

```

```

printf("Deutsch Algorithm\n");

run_deutsch(oracle_const0, "f(x)=0");
run_deutsch(oracle_const1, "f(x)=1");
run_deutsch(oracle_bal_x, "f(x)=x");
run_deutsch(oracle_bal_notx, "f(x)=NOT x");

cleanup_platform();
return 0;
}

```

## 5.2 Grover Search (3 Qubits)

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

#define N 8

void hadamard_all(double *state)
{
    double s = 1.0 / sqrt(2.0);
    double tmp[N];

    for (int i = 0; i < N; i++) {
        tmp[i] = 0;
        for (int j = 0; j < N; j++) {
            int xorv = i ^ j;
            int bits = __builtin_popcount(xorv);
            double sign = (bits % 2 == 0) ? 1 : -1;
            tmp[i] += sign * state[j];
        }
        tmp[i] *= s * s * s;
    }

    for (int i = 0; i < N; i++) state[i] = tmp[i];
}

void oracle_mark(double *state, int target)
{
    state[target] = -state[target];
}

void diffusion(double *state)
{
    hadamard_all(state);

    for (int i = 1; i < N; i++) state[i] = -state[i];
}

```

```

    hadamard_all(state);
}

int main()
{
    init_platform();

    int target = 5;

    double psi[N];
    double init = 1.0 / sqrt(N);

    for (int i = 0; i < N; i++) psi[i] = init;

    int iter = (int)(acos(1.0 / sqrt(N)) / acos(1 - 2.0 / N) +
        0.5);

    for (int k = 0; k < iter; k++) {
        oracle_mark(psi, target);
        diffusion(psi);
    }

    double max = 0;
    int idx = 0;

    for (int i = 0; i < N; i++) {
        double p = psi[i] * psi[i];
        if (p > max) {
            max = p;
            idx = i;
        }
    }

    printf("Measured state: %d\n", idx);
    printf("Probability: %.4f\n", max);

    cleanup_platform();
    return 0;
}

```

### 5.3 3-Qubit Quantum Fourier Transform

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

void had(double *r, double *im, int q)
{

```

```

double s = 1.0 / sqrt(2.0);
double tr[8], ti[8];

for (int i = 0; i < 8; i++) {
    tr[i] = r[i];
    ti[i] = im[i];
}

for (int i = 0; i < 8; i++) {
    int bit = (i >> (2 - q)) & 1;
    if (!bit) {
        int j = i | (1 << (2 - q));
        tr[i] = s * (r[i] + r[j]);
        ti[i] = s * (im[i] + im[j]);
        tr[j] = s * (r[i] - r[j]);
        ti[j] = s * (im[i] - im[j]);
    }
}

for (int i = 0; i < 8; i++) {
    r[i] = tr[i];
    im[i] = ti[i];
}

}

void cphase(double *r, double *im, int c, int t, double th)
{
    double ct = cos(th);
    double st = sin(th);

    for (int i = 0; i < 8; i++) {
        int cb = (i >> (2 - c)) & 1;
        int tb = (i >> (2 - t)) & 1;
        if (cb && tb) {
            double rr = ct * r[i] - st * im[i];
            double ii = st * r[i] + ct * im[i];
            r[i] = rr;
            im[i] = ii;
        }
    }
}

void swap(double *r, double *im, int q1, int q2)
{
    for (int i = 0; i < 8; i++) {
        int b1 = (i >> (2 - q1)) & 1;
        int b2 = (i >> (2 - q2)) & 1;

        if (b1 != b2) {
            int j = i ^ (1 << (2 - q1)) ^ (1 << (2 - q2));
            if (i < j) {

```

```

        double tr = r[i], ti = im[i];
        r[i] = r[j]; im[i] = im[j];
        r[j] = tr;    im[j] = ti;
    }
}

}

int main()
{
    init_platform();

    double pi = 3.1415926535;
    double r[8], im[8];

    for (int i = 0; i < 8; i++) {
        r[i] = (i == 3) ? 1.0 : 0.0;
        im[i] = 0.0;
    }

    had(r, im, 0);
    cphase(r, im, 1, 0, pi / 2);
    cphase(r, im, 2, 0, pi / 4);

    had(r, im, 1);
    cphase(r, im, 2, 1, pi / 2);

    had(r, im, 2);

    swap(r, im, 0, 2);

    for (int i = 0; i < 8; i++) {
        double mag = sqrt(r[i] * r[i] + im[i] * im[i]);
        printf("%d: %.4f\n", i, mag);
    }

    cleanup_platform();
    return 0;
}

```

## 5.4 Quantum Phase Kickback

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

```

```

double s = 1.0 / sqrt(2.0);

double psi[4] = {0.0, s, 0.0, s};

double t[4] = {psi[0], psi[3], psi[2], psi[1]};
for (int i = 0; i < 4; i++) psi[i] = t[i];

printf("After phase kickback:\n");
printf("[%f %f %f %f]\n",
        psi[0], psi[1], psi[2], psi[3]);

cleanup_platform();
return 0;
}

```

## 5.5 Quantum Phase Estimation (Simplified)

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();

    double pi = 3.1415926535;
    int precision = 3;

    double true_phase = 1.0 / 8.0;

    printf("QPE with 3-bit precision\n");
    printf("True phase = %f\n", true_phase);

    for (int k = 0; k < precision; k++) {
        int p = 1 << k;
        double acc = fmod(p * true_phase, 1.0);
        printf("Control qubit %d applies U^(%d), angle = %f\n",
                k, p, 2 * pi * acc);
    }

    int measured = (int)(true_phase * (1 << precision) + 0.5);
    double est = (double)measured / (1 << precision);

    printf("Measurement: %d (%d%d%d)\n",
            measured,
            (measured >> 2) & 1,
            (measured >> 1) & 1,
            measured & 1);
}

```

```

    printf("Estimated phase = %.4f\n", est);
    printf("Error = %.6f\n", fabs(est - true_phase));

    cleanup_platform();
    return 0;
}

```

## 5.6 Variational Circuit (QAOA Example)

```

#include <stdio.h>
#include <math.h>
#include "platform.h"
#include "xil_printf.h"

void mix(double *s, double beta)
{
    double c = cos(beta);
    double si = sin(beta);
    double t[4];

    t[0] = c * s[0] + si * s[1];
    t[1] = -si * s[0] + c * s[1];
    t[2] = c * s[2] + si * s[3];
    t[3] = -si * s[2] + c * s[3];

    for (int i = 0; i < 4; i++) s[i] = t[i];
}

void problem(double *s, double g)
{
    double c = cos(g);

    s[0] *= c;
    s[1] *= c;
    s[2] *= c;
    s[3] *= c;
}

double cost(double *s)
{
    return s[1] * s[1] + s[2] * s[2];
}

int main()
{
    init_platform();

    double s[4] = {0.5, 0.5, 0.5, 0.5};
    double beta = 0.5, gamma = 1.0;
}

```

```

    for (int i = 0; i < 2; i++) {
        problem(s, gamma);
        mix(s, beta);
    }

    double e = cost(s);

    printf("Expectation: %.4f\n", e);

    double max = 0;
    int idx = 0;
    for (int i = 0; i < 4; i++) {
        double p = s[i] * s[i];
        if (p > max) {
            max = p;
            idx = i;
        }
    }

    printf("Most likely state: %d (prob %.4f)\n", idx, max);

    cleanup_platform();
    return 0;
}

```

## 6 Compilation and Testing

### 6.1 Makefile for Zedboard

```

CC = arm-linux-gnueabihf-gcc
CFLAGS = -Wall -O2 -g -std=c99
LDFLAGS = -lm

XILINX_SDK = /opt/Xilinx/SDK/2019.1
BSP_DIR = ./bsp
INCLUDE = -I$(BSP_DIR)/include

SRCS = quantum_sim.c $(BSP_DIR)/platform.c
OBJS = $(SRCS:.c=.o)
TARGET = quantum_sim.elf

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) $(INCLUDE) -c $< -o $@

```

```
clean:
    rm -f $(OBJS) $(TARGET)

deploy:
    xsct deploy.tcl

.PHONY: all clean deploy
```

## 6.2 Challenge Problems

1. Implement 3-qubit bit-flip code.
2. Implement Shor's modular exponentiation block.
3. Implement HHL linear solver.
4. Convert simulation to fixed-point arithmetic.
5. Build hybrid CPU/FPGA optimization loop.

## 7 Performance Optimization

### 7.1 Fixed-Point Arithmetic

```
typedef int32_t fixed_t;
#define FIXED_SHIFT 16
#define FIXED_ONE (1 << FIXED_SHIFT)

fixed_t float_to_fixed(double x)
{
    return (fixed_t)(x * FIXED_ONE);
}

double fixed_to_float(fixed_t x)
{
    return (double)x / FIXED_ONE;
}

fixed_t fixed_mult(fixed_t a, fixed_t b)
{
    return (fixed_t)((((int64_t)a * b) >> FIXED_SHIFT));
}
```

### 7.2 FPGA Resource Notes

Approximate resource usage for a 3-qubit simulator:

- LUTs: 5000

- FFs: 4000
- DSP slices: minimal if using fixed-point
- BRAM: small usage for state vector